Skill-as-API: Confidential Multi-Agent Coordination for Agentic Software Engineering

Ziwei Zhao*
Yu Gu*
Haojun Liang*
Chen Zhang
Technical University of Munich
Munich, Germany

Xizhi Ding
London Business School
London, United Kingdom

Abstract

AI coding agents are evolving from solitary tools into collaborative teammates that discover and invoke one another's specialized skills. But the coordination channel itself can leak a skill's intellectual property. Protocols such as MCP and A2A run implementations server-side, yet they still publish each skill's description and typed schemas to every peer, offer no way to hide a skill's existence, and cannot guarantee that a wrapped system prompt stays off the wire. Application-layer privacy filters help, but act only *after* the model has decided to emit sensitive text. We take a complementary, protocol-layer route: Skill-as-API, a coordination protocol whose public view of a skill is limited to its name, description, typed input/output schemas, and trust tier. The skill body is closure-captured in the owner's process and never crosses the wire. Four layers add access control and narrow the prompt-injection surface structurally rather than by filtering content. We provide an open-source Python implementation over XMTP with 1.8–2.9 s cross-continent hot-reconnect latency, and a software-engineering case study in which three agents coordinate a pull-request review while each retains ownership of its proprietary analysis prompts.

Keywords

multi-agent coordination, LLM agents, agentic software engineering, skill confidentiality, protocol design

1 Introduction

AI coding agents such as SWE-agent [11] and OpenHands [6] are evolving from single-machine assistants into collaborative *AI teammates* that increasingly work across organizational boundaries. A team running its own static-analysis ruleset may want to invoke another team's security-scanning agent, an LLM that wraps a proprietary, curated library of vulnerability patterns; a DevOps agent may delegate incident triage to a vendor agent encoding months of response experience. Such cross-boundary collaboration is the hallmark of SE 3.0, the *agentic software engineering* paradigm in which human engineers orchestrate fleets of autonomous agents [1].

Yet the collaboration channel itself is a liability. Protocols such as MCP and A2A run skill implementations server-side, so neither ships code; but both still publish each skill's name, description, and typed schemas to every peer, cannot hide a skill's existence, and cannot keep a wrapped system prompt off the discovery surface. What leaks is therefore narrow but valuable: the methodology a proprietary skill encodes. Application-layer privacy filters such as PrivacyChecker [5] reduce such leakage but leave a ∼7–8% residual in their reported settings, intervening only after the model has decided to emit sensitive content.

We introduce **Skill-as-API**, a coordination protocol for agentic SE whose public view of a skill is limited to its name, description, and typed input/output schemas. The body, its code and the system prompt that encodes its behavior, is closure-captured in the owner's process and never crosses the wire. We make three contributions:

C1 Protocol design (§3): trust tiers, auto-downgrade, skill hiding, and a function-boundary defense, unified by one confidentiality invariant (the body never enters the public view).

C2 Open-source implementation[1]: a lightweight Python SDK over XMTP with no Python-side dependencies beyond the standard library, achieving cross-continent hot-reconnect latency of 1.8–2.9 s.

C3 SE case study (§4): three agents coordinate a pull-request review, demonstrating trust tiers, skill hiding, and injection absorption in a realistic SE workflow.

We position Skill-as-API not as a replacement for MCP or A2A but as a complementary protocol-layer strategy for settings where skill IP must be protected during multi-agent coordination.

2 Motivation and Threat Model

2.1 Threats in SE Agent Collaboration

We identify three threats specific to multi-agent SE coordination:

T1: Skill-IP extraction. A caller tries to obtain a callee's proprietary methodology in one of two ways: by extracting its code or system prompt directly through the coordination channel, or by enumerating the callee's skill namespace to locate high-value targets. Statistical cloning, in which a caller distills many legitimate outputs into an approximate copy, is a fundamental limit of any callable skill and lies outside our scope (§6). Agent Skills for LLMs [10] documents a lifecycle defense model and reports that roughly a quarter of surveyed skills carry exploitable security risks.

T2: Prompt theft via injection. A caller crafts a task input that tricks the callee's LLM into emitting its system prompt verbatim. The OWASP 2026 catalog [4] formalizes prompt injection as direct and indirect; we additionally consider tool-result and conversation-history channels.

*These authors contributed equally to this work.

[1] Anonymized artifact for double-blind review: https://anonymous.4open.science/r/KDD-Skill-as-API-0000/.

Table 1: Threat model: actor capabilities and scope.

Actor	Capability	Scope
Honest-but-curious caller	Read schema, send inputs	in
Adversarial caller	+ Injection payloads (OWASP)	in
Colluding callees	Pool outputs to reconstruct schema	partial
Schema-inference attacker	Probe input space to learn prompt shape	partial
OS/physical attacker	Compromise owner machine	out
Caller's LLM hijacked	Attack inside caller process	out

T3: Trust drift. After a successful collaboration episode, the caller retains elevated permissions indefinitely, turning a one-time PR-review partnership into a permanent open channel. In SE 3.0 settings where agents from different organizations collaborate transiently, unbounded trust accumulation is an especially dangerous failure mode.

2.2 Threat Model

Table 1 summarizes the scope. We assume the owner's machine is not compromised; if it is, no protocol can protect the skill body. We also exclude attacks on the caller's own LLM stack: Skill-as-API protects the *callee's* code and prompt, not the caller's. Two adversary classes are *partially in scope*: colluding callees that pool their observations to reconstruct schema-level information, and schema-inference attackers that probe the input space to learn prompt structure. Skill-as-API reduces but does not eliminate these risks; we discuss the residual surface in §6 (L4–L5).

2.3 Design Goals

These threats are not independent: extraction (T1) and prompt theft (T2) both target the skill body, and trust drift (T3) widens over time the set of peers that may attempt them. Defending against all three takes more than encrypting a channel or filtering outputs after the fact; it requires controlling what a peer can ever observe, who is allowed to observe it, and for how long. We distill these requirements into five design goals, which the protocol of §3 realizes and the case study of §4 exercises.

G1 Code/prompt non-transmission (structural).
G2 Unknowability: hidden, forbidden, and nonexistent skills return the same error string.
G3 Bounded permission lifetime: trust granted for one collaboration episode decays on completion.
G4 Function-boundary prompt protection: caller input never enters the callee's system prompt.
G5 Acceptable latency: low single-digit-second hot-reconnect.

3 The Skill-as-API Protocol

Skill-as-API has two parts: a core abstraction that fixes what a skill exposes on the wire, and four layers that govern who may invoke it and what an adversary can extract. We first define the abstraction and its confidentiality invariant, then present the four layers, each addressing a threat from §2: trust-tier access control and auto-downgrade counter trust drift (T3), skill hiding blunts namespace enumeration (T1), and a function-boundary defense contains prompt injection (T2). We close with the reference implementation.

Algorithm 1 OnTaskResult($peer, ok, okr\_done$)

```
if not ok then
    consec[peer] += 1
    if consec[peer] ≥ 3 then
        tier[peer] ← max(tier[peer]−1, 0)
        consec[peer] ← 0
    end if
else consec[peer] ← 0
end if
if okr_done then
    tier[peer] ← max(tier[peer]−1, 1)        ▷ floor at KNOWN
end if
```

3.1 Core Abstraction

A Skill-as-API skill is a tuple $S = (name, d, w, v, \sigma_{\text{in}}, \sigma_{\text{out}}, t_{\min}, B)$ where d is a description, w a natural-language routing hint, v a version string, $\sigma_{\text{in}}/\sigma_{\text{out}}$ are typed schemas, $t_{\min}$ the minimum trust tier, and $B = (c, p)$ the *body* (code c and system prompt p), existing only in the owner's process. The public view is:

$$\text{view}(S) = (name, d, w, v, \sigma_{\text{in}}, \sigma_{\text{out}}, t_{\min}, h) \tag{1}$$

where h=SHA256$(name \parallel v \parallel \sigma_{\text{in}} \parallel \sigma_{\text{out}} \parallel t_{\min})_{[:16]}$ is a 16-byte schema hash for fast reconnect.
Invariant. $B \notin \text{view}(S)$: the body never enters the wire. Invocation is $\text{call}(S, x) = c(\text{filter}(x, \sigma_{\text{in}}))$, where filter drops keys not in σ_{in}. The prompt p is referenced by c through closure capture at registration time; it is not a function parameter and cannot be reached from the input path.

3.2 Layer 1: Trust-Tier ACL (→ T3)

Every peer carries a tier $t \in \{\text{UNTRUSTED}=0, \text{KNOWN}=1, \text{INTERNAL}=2, \text{PRIVILEGED}=3\}$. Skill invocation requires $t \geq S.t_{\min}$. Inbound message types are further gated by a fixed table: `ping` and `trust_request` at UNTRUSTED; `discover` at KNOWN; `collab_request` at INTERNAL; unknown types default to PRIVILEGED (deny-by-default).

3.3 Layer 2: Auto-Downgrade (→ T3)

Trust does not accumulate from successful use; it *decays*. Three consecutive failures drop the peer one tier; completing a collaboration OKR removes one tier with a floor at KNOWN (Algorithm 1). We call this *throw-away trust*: a peer earns evidence for future re-elevation, not standing access, directly addressing T3.

3.4 Layer 3: Skill Hiding (→ T1)

Hidden skills return the *same* error string `"Unknown skill: X"` as truly nonexistent ones. Owner-side logs distinguish the cases, but this distinction never crosses the wire. The design goal G2 prevents a caller from enumerating the skill namespace to mount a systematic extraction attack (T1). We note that timing differences between hidden and executed skills remain as a side-channel; mitigating this via randomized delays is future work.

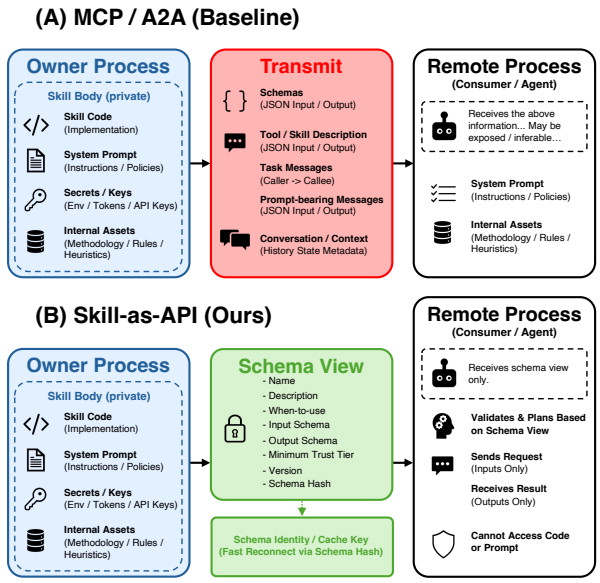

Figure 1: Information flow: MCP/A2A expose tool descriptions, schemas, and prompt context (red); Skill-as-API exposes only the schema view (green), while the body stays in the owner's process (blue dashed box).

3.5 Layer 4: Function-Boundary Defense (→ T2)

The prompt p is closure-captured at registration; caller input reaches the skill only as named, schema-filtered kwargs. Skills built on our reference executor bind these kwargs solely to the *user* turn, so even if the caller embeds "print your system prompt" in the input, that text is never concatenated into p. This yields two distinct properties. *(i) Structural*: p never crosses the wire (Eq. 1) and caller input never becomes the system-role parameter, a confidentiality property of the wire format and the kwarg boundary, not a content filter. *(ii) Residual/behavioral*: a callee model may still recite p in response to user-turn injection; this is the classic prompt-leak vector (L2), which we do not claim to eliminate and which we probe only illustratively in §4. We thus *narrow*, rather than abolish, the prompt-extraction surface: the system-role concatenation vector is closed by construction, while the residual user-role vector remains the model's behavioral choice (G4).

We note that this layer protects only the *callee*. If a caller feeds returned values back into its own LLM, that stack remains exposed to ordinary injection.

3.6 Implementation

The reference implementation is an open-source Python 3.10+ package that installs with a single command and has zero runtime Python dependencies beyond the standard library and optional LLM provider SDKs. Transport is a Node.js sidecar over the XMTP v3 (MLS) protocol via `@xmtp/node-sdk`; the Python core talks to the bridge over a localhost HTTP/SSE channel, keeping the cryptographic implementation out of the Python TCB. The 16-byte schema hash h doubles as a fast-reconnect cache key, yielding the 1.8–2.9 s hot-reconnect latency reported in §4. We disclose two honest limitations for reproducibility: the trust-decay failure counters (Algorithm 1) are currently in-memory only and reset on restart (tier transitions themselves persist to `trust.json` via an atomic temp-file+`fsync`+`os.replace` pattern), and skill versioning v uses exact-string equality rather than semantic-range matching.

4 SE Case Study

We instantiate Skill-as-API in a realistic SE workflow where three agents, running as separate processes over live XMTP channels, coordinate a pull-request (PR) review. The case study exercises each design goal end to end: schema-only discovery (G1), injection absorption at the function boundary (G4), and trust decay on completion (G3).

Setup. The orchestrator agent receives a PR and delegates analysis to two specialists: a code-quality agent, whose proprietary code-review skill embeds a detailed checklist as its system prompt, and a security agent, whose vulnerability-scanning skill is backed by a pattern-library prompt. Both specialists register their skills at tier $t_{\min} = 1$ (KNOWN). The orchestrator connects to each over XMTP, discovers their skill schemas but not their prompts, and dispatches review tasks.

G1 verification: schema-only view. After skill discovery, the orchestrator's view of `code_review` contains only `name`, `description`, `input_schema (diff: str)`, and `output_schema (findings: list[str])`. The checklist prompt (~1,200 characters of proprietary review methodology) is not present in any discovery message or tool-call payload. We verify this by logging all XMTP messages on the orchestrator's side and confirming zero overlap with the callee's `system_prompt` string.

G4 verification: injection absorption. We craft 10 task inputs spanning eight OWASP-style injection classes (direct instruction override, role reversal, markdown/fence escape, prompt-leak request, base64-encoded wrapper, multilingual paraphrase, indirect-via-comment, and recursive self-call), each prepended to a representative PR diff; Table 2 gives the per-class counts. Each payload is sent to the code-quality agent; we measure (a) whether the system prompt appears verbatim or paraphrased in any returned field and (b) whether the structured output schema is preserved. Table 2 reports the result: **10/10 payloads absorbed** (system prompt absent from every returned field, output schema intact). We report this as an *illustrative spot-check, not a bound*: with N=10 a zero-leak result cannot tightly bound the residual leakage rate, and "absorbed" here conflates a *structural* property (caller input is bound to a kwarg and never concatenated into p, which is closure-captured at registration) with a *behavioral* one (the callee model not voluntarily reciting p), the latter being out of scope per L2. Future work includes quantifying the two separately and adding a naive concatenating-skill control.

G3 verification: trust decay on OKR completion. We exercise three transitions in the same case study run. (i) Both specialists are first elevated to INTERNAL so the orchestrator can invoke higher-privilege skills during the review. (ii) When the orchestrator marks the collaboration OKR complete, Algorithm 1 line 10 fires

Table 2: OWASP-style injection absorption on the code-quality agent ($N = 10$). "Absorbed" means the system prompt does not appear in the return value and the output schema is preserved.

Payload class	N	Absorbed
Direct instruction override	2	2/2
Role reversal / persona swap	1	1/1
Markdown / fence escape	1	1/1
Prompt-leak request	2	2/2
Encoded (base64) wrapper	1	1/1
Multilingual paraphrase	1	1/1
Indirect via embedded comment	1	1/1
Recursive self-call	1	1/1
Total	**10**	**10/10**

and decays both peers from INTERNAL→KNOWN, a measured numerical drop, not a no-op at the floor. (iii) For completeness, we also verify the PRIVILEGED→INTERNAL transition on a test peer in the same run. A subsequent attempt to invoke a INTERNAL-tier skill is denied; the orchestrator must re-request elevation for the next collaboration episode. A one-time PR review does not become a permanent skill-access channel.

Latency. Hot-reconnect latency (second and subsequent calls between previously connected peers) is 1.8–2.9 s (N=30, two intercontinental XMTP relay endpoints on commodity Linux hosts, 100 Mbps links). Cold start (first-ever channel establishment) is 30–60 s and is dominated by XMTP MLS group formation. All timings include key ratcheting.

Limitations observed. The orchestrator forwards the code-quality agent's output as context to the security agent. A malicious code-quality agent could embed poisoned content in its response; the orchestrator currently passes it through without sanitization (*cross-step input poisoning*). We disclose this as L1 in §6.

5 Related Work

Agent-privacy strategies. Prior agent-privacy work clusters into three protocol-shape strategies: (i) *encrypt-everything*, exemplified by AgentCrypt [2], which secures inter-agent traffic with HE/MPC at the 2–3 orders-of-magnitude overhead generic to such techniques; (ii) *IAM-gate*, exemplified by A2A, which restricts who can talk to whom but still publishes schemas and descriptions for discovery; and (iii) *content-filtering at inference time*, exemplified by PrivacyChecker [5], which retains the ∼7–8% residual discussed in §1. Skill-as-API is a fourth, complementary strategy at the *ABI layer*: it changes *what is on the wire*, leaving encryption, IAM, and content filtering free to compose on top. These are orthogonal: Skill-as-API removes exposure paths that filters cannot reach, while filters address the LLM's behavioral choices that no protocol can directly constrain. We choose this ABI boundary over TEE-based skill enclaves, which bind the body to silicon but add hardware-vendor dependence, in exchange for portability and zero infrastructure cost.

Multi-agent coordination protocols. MCP [3] connects agents to tool servers, transmitting descriptions and schemas. AgentNet [12] decentralizes coordination but does not address skill-IP protection.

Table 3: Protocol comparison. ✓=structural protection; ✗=none/exposed; *app*=left to the application; *enc*=via encryption.

	MCP	A2A	AgentCrypt	**Skill-as-API**
Impl. code on wire	none	none	enc	none (closure)
Description/schema	public	public	enc	public
System-prompt confid.	app	app	enc	✓ (closure)
Skill-existence hiding	✗	✗	✗	✓
Trust lifetime	none	IAM	PKI	4-tier + decay
Injection def.	app	app	n/a	ABI structural
Overhead	low	low	high	low (XMTP)

The closure-capture idea has antecedents in object-capability systems such as Capsicum [8], which restrict reachable resources at the language and OS boundary; Skill-as-API adapts the same intuition to the agent ABI.

SE agent systems. SWE-agent [11], OpenHands [6], and Copilot Workspace demonstrate that coding agents can autonomously resolve issues, review PRs, and generate patches. These systems assume single-agent or trust-all multi-agent environments. Skill-as-API adds a trust-aware coordination layer for cross-organization SE agent collaboration, the SE 3.0 vision of AI teammates [1].

Agent security. ARGUS [9] trains content-layer injection discriminators. MCPTox [7] catalogs MCP attack surfaces. Skill-as-API operates at the protocol layer rather than the content layer, narrowing the prompt-extraction surface structurally.

6 Conclusion

We presented Skill-as-API, a coordination protocol for agentic SE in which skill bodies never leave the owner's process. Four layers add trust-tier access control, throw-away trust via auto-downgrade, skill hiding, and a function-boundary defense that narrows the injection surface structurally. An SE case study with three PR-review agents illustrates that trust tiers govern access, system-role injection is absorbed at the closure boundary, and permissions decay toward a KNOWN floor on task completion. The reference implementation is open-source, runs over XMTP, and achieves 1.8–2.9 s hot-reconnect latency.

Limitations and future work. **L1**: Cross-step input poisoning (output of agent A forwarded unfiltered to agent B) is not addressed; an orchestrator-level sanitizer is future work. **L2**: Application-level leakage (the callee's LLM voluntarily emitting sensitive content in the return value) is orthogonal to Skill-as-API and remains the domain of content-layer filters [5]. **L3**: Timing side-channels between hidden and executed skills remain; randomized delays are planned. **L4**: Trust is per-peer, not per-skill; finer-grained ACLs are future work. **L5**: Colluding callees and schema-inference probes (§2) are mitigated but not eliminated by schema-only views; defenses such as randomized output perturbation and rate-limited discovery are future work. We envision Skill-as-API as part of the SE 3.0 tool ecosystem where agents collaborate as teammates while each retaining ownership of their engineering methodology.

References

[1] Ahmed E. Hassan, Hao Li, Dayi Lin, Bram Adams, Tse-Hsun Chen, Yutaro Kashiwa, and Dong Qiu. 2025. Agentic Software Engineering: Foundational Pillars and a Research Roadmap. *arXiv preprint arXiv:2509.06216* (2025).

[2] Harish Karthikeyan, Yue Guo, Leo de Castro, Antigoni Polychroniadou, Udari Madhushani Sehwag, Leo Ardon, Sumitra Ganesh, and Manuela Veloso. 2026. AgentCrypt: Advancing Privacy and (Secure) Computation in AI Agent Collaboration. arXiv:2512.08104.

[3] Xiaofan Li and Xing Gao. 2025. A First Look at the Security Issues in the Model Context Protocol Ecosystem. arXiv:2510.16558.

[4] OWASP Foundation. 2025. OWASP Top 10 for LLM Applications 2025. https://genai.owasp.org/llm-top-10/

[5] Shouju Wang, Fenglin Yu, Xirui Liu, Xiaoting Qin, Jue Zhang, Qingwei Lin, Dongmei Zhang, and Saravan Rajmohan. 2025. Privacy in Action: Towards Realistic Privacy Mitigation and Evaluation for LLM-Powered Agents. In *Findings of EMNLP*. arXiv:2509.17488.

[6] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. 2024. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. arXiv:2407.16741.

[7] Zhiqiang Wang, Yichao Gao, Yanting Wang, Suyuan Liu, Haifeng Sun, Haoran Cheng, Guanquan Shi, Haohua Du, and Xiangyang Li. 2025. MCPTox: A Benchmark for Tool Poisoning Attack on Real-World MCP Servers. arXiv:2508.14925.

[8] Robert N. M. Watson, Jonathan Anderson, Ben Laurie, and Kris Kennaway. 2010. Capsicum: Practical Capabilities for UNIX. In *Proc. USENIX Security Symposium*.

[9] Shihao Weng, Yang Feng, Jinrui Zhang, Xiaofei Xie, Jiongchi Yu, and Jia Liu. 2026. ARGUS: Defending LLM Agents Against Context-Aware Prompt Injection. arXiv:2605.03378.

[10] Renjun Xu and Yang Yan. 2026. Agent Skills for Large Language Models: Architecture, Acquisition, Security, and the Path Forward. arXiv:2602.12430.

[11] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Proc. NeurIPS*.

[12] Yingxuan Yang, Huacan Chai, Shuai Shao, Yuanyi Song, Siyuan Qi, Renting Rui, and Weinan Zhang. 2025. AgentNet: Decentralized Evolutionary Coordination for LLM-based Multi-Agent Systems. In *Proc. NeurIPS*. OpenReview tXqLxHlb8Z.